

```

64 var x = o.left;
65 var y = o.top;
66
67 var ax = settings.accX;
68 var ay = settings.accY;
69 var th = t.height();
70 var wh = w.height();
71 var tw = t.width();
72 var ww = w.width();
73
74 if (y + th + ay >= b &&
75     y <= b + wh + ay &&
76     x + tw + ax >= a &&
77     x <= a + ww + ax) {
78     //trigger the custom event
79     if (!t.appeared) t.trigger('appear', settings.data);
80
81     } else {
82     //it scrolled out of view
83     t.appeared = false;
84
85     }
86
87 };
88
89 //create a modified fn with some additional logic
90 var modifiedFn = function() {
91
92     //mark the element as visible
93     t.appeared = true;
94
95     //is this supposed to happen only once?
96     if (settings.one) {
97
98         //remove the check
99         w.unbind('scroll', check);
100         var i = $.inArray(check, $.fn.appear.checks);
101         if (i >= 0) $.fn.appear.checks.splice(i, 1);
102
103     }
104
105     //trigger the original fn
106     fn.apply(this, arguments);
107
108 };
109
110 //bind the modified fn to the element
111 $(settings.one) t.one('appear', settings.data, modifiedFn);
112 settings.data, modifiedFn);

```



MATERIA:

Programación Orientada a Objetos

Unidad 2 Principios de la Programación Orientada a Objetos.

Docente:

Grupo:

Unidad #2 - Principios de la Programación Orientada a Objetos.

2.4. Polimorfismo.

- 2.4.1. Asignación de objetos a variables de su superclase.

2.5. Interfaces.

- 2.5.1. Definición e implementación de interfaz.
- 2.5.2. Interfaces y polimorfismo.

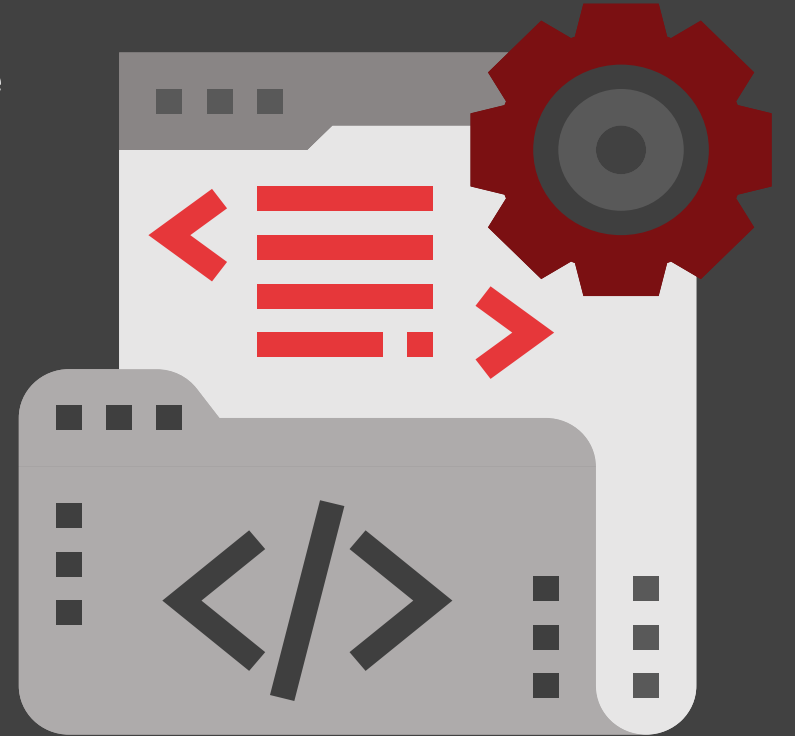
2.6. Clases y métodos abstractos

2.4 Polimorfismo

Polimorfismo = “muchas formas”.

En **programación orientada a objetos**, significa que un mismo mensaje (o método) puede producir **diferentes comportamientos** dependiendo del objeto que lo ejecute.

En pocas palabras: **el mismo método, distintas respuestas.**



2.4 Polimorfismo

Concepto reforzado

- **Tipo de referencia** (**Animal**): define qué métodos *pueden* llamarse.
- **Tipo de objeto real** (**Perro** o **Gato**): define qué implementación se ejecuta.
- Resultado: un **mismo método** (**hacerSonido**) produce **diferentes comportamientos** según el objeto → esto es **polimorfismo**.

```
class Animal { 4 usages 2 inheritors new *
    void hacerSonido() { System.out.println("Sonido genérico"); }
}

class Perro extends Animal { 1 usage new *
    @Override void hacerSonido() { System.out.println("Guau!"); }
}

class Gato extends Animal { 1 usage new *
    @Override void hacerSonido() { System.out.println("Miau!"); }
}

public class Demo { 2 franciscomorales25 *
    public static void main(String[] args) { new *
        Animal a1 = new Perro();
        Animal a2 = new Gato();

        a1.hacerSonido(); // Guau!
        a2.hacerSonido(); // Miau!
    }
}
```

Ventajas del Polimorfismo

- Código **flexible** y **genérico**.
- Se pueden agregar nuevas clases sin cambiar el código existente.
- Evita condicionales largos (**if**, **switch**).
- Permite trabajar con colecciones de objetos distintos bajo un mismo tipo.

2.4.1. Asignación de objetos a variables de su superclase

En Java, una variable de **superclase** puede **referenciar objetos de cualquier subclase**.

A esto se le llama **upcasting** (conversión hacia arriba).

Es **automático y seguro**, porque toda subclase **es también** un tipo de su superclase.

```
class Animal { 3 usages 2 inheritors new *
    System.out.println("Sonido genérico");
}

class Perro extends Animal { 1 usage new *
    @Override no usages new *
    void hacerSonido() {
        System.out.println("Guau!");
    }
}

class Gato extends Animal { 1 usage 2 franciscomorales25 *
    @Override no usages new *
    void hacerSonido() {
        System.out.println("Miau!");
    }
}
```

```
public class Demo { 2 franciscomorales25 *
    public static void main(String[] args) { new *
        Animal miAnimal; // variable de tipo superclase

        miAnimal = new Perro(); // asignación de objeto Perro
        miAnimal.hacerSonido(); // Guau!

        miAnimal = new Gato(); // asignación de objeto Gato
        miAnimal.hacerSonido(); // Miau!
    }
}
```


2.4.1. Asignación de objetos a variables de su superclase

Restricción importante

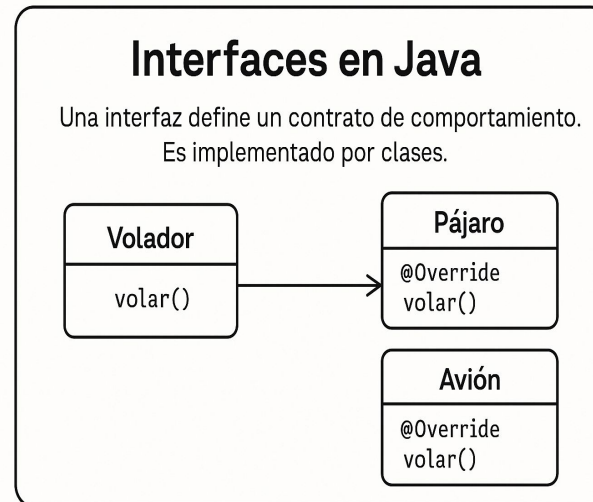
- La variable de superclase **solo puede acceder** a los métodos definidos en la superclase.
- No puede usar métodos propios de la subclase, a menos que se haga un **downcasting**.

```
Perro p = new Perro(); no usages new *  
p.moverCola(); // válido si existe en Perro new *  
  
Animal a = new Perro(); 1 usage  
// a.moverCola(); // ✗ error: no está definido en Ani  
((Perro)a).moverCola(); // ✓ downcasting new *
```

2.5. Interfaces.

Concepto de Interfaz

- Una **interfaz** es un **contrato de comportamiento**.
- Define **qué métodos debe tener** una clase, pero no cómo implementarlos.
- Se declara con la palabra clave **interface**.
- Una clase que implementa la interfaz está **obligada** a sobrescribir sus métodos.



Características principales

- Métodos → **public abstract** por defecto.
- Atributos → siempre **public static final** (constantes).
- Una interfaz **no se puede instanciar**.
- Una clase puede **implementar varias interfaces**.
- Facilitan el **polimorfismo**.

Ejemplo de definición e implementación

```
interface Volador { 2 usages 2 implementations new *  
    void volar(); no usages 2 implementations new *  
}  
  
class Pajaro implements Volador { no usages 2 franciscomorales25 *  
    @Override no usages new *  
    public void volar() {  
        System.out.println("El pájaro vuela con sus alas");  
    }  
}  
  
class Avion implements Volador { no usages 2 franciscomorales25 *  
    @Override no usages new *  
    public void volar() {  
        System.out.println("El avión vuela con sus motores");  
    }  
}
```

2.5.2 Interfaces y polimorfismo

Concepto

- El **polimorfismo con interfaces** permite que distintas clases se manejen bajo un **mismo contrato**.
- Una **variable de tipo interfaz** puede almacenar cualquier objeto que la implemente.
- El método que se ejecuta depende del **objeto real**.

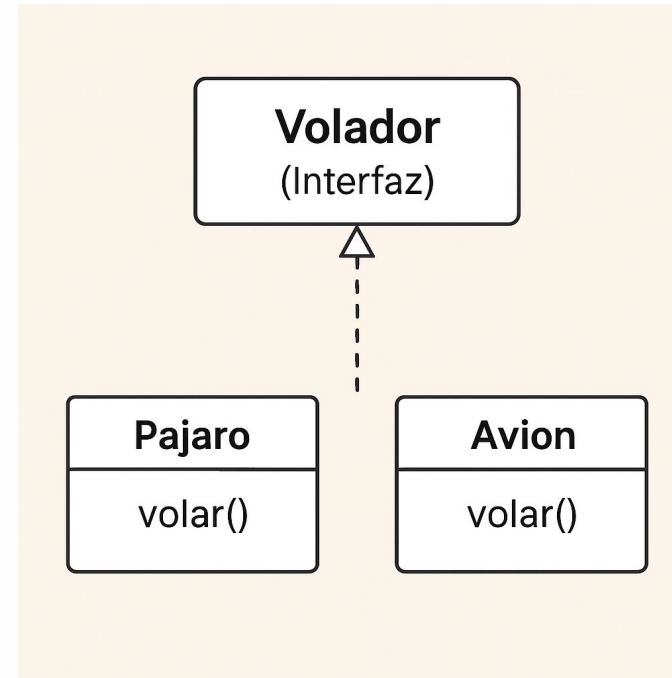
```
interface Volador { 4 usages 2 implementations new *  
    void volar(); 2 usages 2 implementations new *  
}  
  
class Pajaro implements Volador { 1 usage new *  
    public void volar() { System.out.println("El pájaro vuela con sus alas"); } 2 usages new *  
}  
  
class Avion implements Volador { 1 usage new *  
    public void volar() { System.out.println("El avión vuela con sus motores"); } 2 usages ne  
}  
  
public class Demo { 2 franciscomorales25 *  
    public static void main(String[] args) { new *  
        Volador v1 = new Pajaro();  
        Volador v2 = new Avion();  
  
        v1.volar(); // El pájaro vuela...  
        v2.volar(); // El avión vuela...  
    }  
}
```

Misma referencia (**Volador**) → distintos comportamientos.

2.5.2 Interfaces y polimorfismo

Ventajas

- **Flexibilidad:** diferentes clases no relacionadas pueden trabajar juntas.
- **Polimorfismo:** mismo contrato, diferentes respuestas.
- **Extensibilidad:** se agregan nuevas clases sin modificar código existente.
- **Desacoplamiento:** el código depende de la interfaz, no de la clase concreta.



2.6. Clases y métodos abstractos

Concepto de Clase Abstracta

- Una **clase abstracta** es una **plantilla base**.
- Puede contener:
 - Métodos abstractos (sin implementación).
 - Métodos concretos (con código).
- **No puede instanciarse directamente.**
- Se declara con la palabra clave **abstract**

Concepto de Método Abstracto

- Declarado con la palabra **abstract**.
- Solo define la firma (nombre + parámetros).
- **No tiene cuerpo** { }.
- Obliga a las subclases a sobrescribirlo.

```
abstract class Figura { no usages new *  
    public abstract double calcularArea(); no  
}
```

2.6. Clases y métodos abstractos

```
abstract class Animal { 2 usages 2 inheritors @ franciscomorale
    abstract void hacerSonido(); no usages new *

    void dormir() { no usages new *
        System.out.println("El animal duerme");
    }
}

class Perro extends Animal { no usages new *
    void hacerSonido() { System.out.println("Guau!"); }
}

class Gato extends Animal { no usages @ franciscomorales25 *
    void hacerSonido() { System.out.println("Miau!"); }
}
```

```
public class Demo { @ franciscomorales25 *
    public static void main(String[] args) { new *
        // Animal a = new Animal(); // ✗ No permitido
        Animal perro = new Perro();
        Animal gato = new Gato();

        perro.hacerSonido(); // Guau!
        gato.hacerSonido(); // Miau!
        perro.dormir(); // Método concreto heredado
    }
}
```

No se puede crear un objeto `Animal` porque es abstracto.
Se crean objetos de las subclases (`Perro`, `Gato`).